

Earthquake Analytics Platform Reflective Report



Reflective Report: Earthquake Web Application Project

Render Deployment url: <https://equaketracker.onrender.com/>

Introduction

Building this application helped me understand how full-stack web systems are made, from design to going online. I learned how backend code, databases, and user interfaces work together. Planning and organizing a project is crucial before coding. Success in programming comes from creating well-organized systems through practice and problem-solving.

Guiding Principle and Approach

The guiding principle that informed my approach to development during creation of the web page was incremental and modular development coupled with testing and committing the work after completion of each iteration. It is important to develop small components which can later be incorporated into the entire program. In particular, for instance, I first created the database design, the import scripts, and lastly the interface.

This method was supplemented with version control through Git whereby individual functions were committed separately to allow easy reverting to previous versions in case of errors. The best practice when coding on Git encourages frequent committing to allow for easier tracking of errors (Atlassian Git Tutorials, 2025). In this particular instance, for instance, I managed to find out what caused the pagination to break the dashboard by reverting to an earlier commit.

The development approach worked well since it simplified my work by reducing complexity. However, it required commitment on my part to avoid doing large chunks of work at once.

Web App Development

I built an application using Flask for the backend and a database to store earthquake information. I organized the code into models, routes, and templates to keep it simple and structured. This setup makes it easier to fix issues or add new features later, with efficient data handling through

Earthquake Analytics Platform Reflective Report

pagination and filtering. Setting up the Flask routes and connecting the database was smooth, but I faced challenges when switching from SQLite to PostgreSQL and managing environment variables. Overall, it was a valuable learning experience in real-world deployment and problem-solving. The completion of this project has helped me gain new knowledge, both technically and personally. In terms of my technical abilities, I gained experience in implementing several different pieces of technology in one piece of software, such as Flask, SQLAlchemy, and Pandas. For instance, I have developed an endpoint that provides earthquake data in JSON form.

Flask Development Challenges

Using Flask had its benefits but also came with some challenges. The Flask framework is highly customizable and lightweight, which allowed me to design my application rapidly. At the same time, because it is customizable, I needed to configure everything on my own, including the router, database connection, and environment variables.

Database configuration was another significant problem. Moving from SQLite database while developing to PostgreSQL while deploying created several configuration issues, which happens frequently with Flask projects due to the migration from development to production (Real Python, 2025). For instance, improper configuration of the environment variable led to an error upon the application's deployment.

Finally, there was a problem with the speed of operations with large databases since without implementing pagination, the application was not performing efficiently. Using server-side pagination helped to solve this problem and optimize the number of queries to the database. SQLAlchemy's documentation says that efficient database queries are important for optimization (SQLAlchemy, 2026).

My Changing Perception of Advanced Programming

When I started this course, I thought advanced programming was just about complex algorithms and fast coding. However, my view changed as I worked on real projects. I learned that advanced programming is about building systems, organizing data, and integrating different technologies. I realized that success in programming involves design, problem-solving, and understanding frameworks and databases. Encountering real issues highlighted the need for planning, structure, and adaptability in programming.

Earthquake Analytics Platform Reflective Report

My Opinion

Overall, I believe that the Advanced programming course (CS551P) focuses on creating well-organized systems that are dependable and easy to grow. It involves planning, integrating different components smoothly, and solving problems throughout the system. Programming combines clear thinking, good design, and continuous learning to build impactful applications. This change of perspective was brought about by experience. Real work with actual data and application scenarios made me realize that the problems encountered have more to do with the configuration and architecture than faulty logic. This further cemented the thought that software engineering is a multidimensional activity.

If I were to rebuild this application, I will first create a detailed system architecture covering database relationships, API design, and deployment. Next, I will implement automatic testing early in development to catch issues quickly.

Conclusion

Working on the Flask earthquake application helped me learn advanced programming concepts. I discovered that quality software needs planning and process improvement, and I developed new problem-solving skills.

References

- Atlassian (2025) *Git Tutorials and Best Practices*. Available at: <https://www.atlassian.com/git/tutorials>
- (Accessed: 15 May 2026).
- Miguel Grinberg (2018) *Flask Web Development: Developing Web Applications with Python*. 2nd edn. Sebastopol: O'Reilly Media.
- SQLAlchemy (2026) *SQLAlchemy Documentation*. Available at: <https://www.sqlalchemy.org/>
- Pandas Development Team (2026) *Pandas Documentation*. Available at: <https://pandas.pydata.org/docs/>